Foreword
Join the Community
Guide Notes Use To Without Limits
Rest of the architecture Exercises 2
A Framework For System Design Interviews
Design a Video Lines
Design Collaborative Editing
Design a Key-value Store
Design a Ticketing Service
Design a Distributed Message Queue
Design a Search Autocomplete System
Design a Web Crawler
Design a Notification System
Design a Newsfeed System
Design a Chat System
Design a Search Autocomplete System
Design YouTube
Design Google Drive
Proximity Service
Nearby Friends
Google Maps
Distributed Message Queue
Metrics Monitoring and Alerting System
Ad Click Event Aggregation
Hotel Reservation System
Distributed Email Service
S3 like Object Storage
Real time Gaming Leaderboard
Payment System
Digital Wallet
Stock Exchange
The Learning Continues

13
Design A Chat System

In this chapter we explore the design of a chat system. Almost everyone uses a chat app. Figure 1 shows some of the most popular apps in the industry.

Figure 1

A chat app performs different functions for different people. It is extremely important to nail down the exact requirements. For example, you do not want to design a system that focuses on group chat when the interviewer has in mind one-to-one chat or vice versa. It is important to explore the feature requirements.

Step 1- Understand the problem and establish design scope

It is vital to agree on the type of chat we plan to design in the marketplace, there are one-on-one chat apps like Facebook Messenger, WeChat and Whatsapp that focuses on one-to-one chat. There are also chat apps that focus on group chat like Slack, or games chat apps, like Discord, that focus on large group interaction and low voice chat latency.

The chat app we plan to design in this chapter should have at a minimum the ability to chat privately with one other user, or to design a chat system, at the very least, the system should have at a minimum chat in group chat app. Some questions you might ask are as follows:

Candidate: What kind of chat app shall we design? 1 on 1 group based?
Interviewer: It should support both 1 on 1 and group chat.
Candidate: Is this a mobile app? Or a web app? Or both?
Interviewer: Both.
Candidate: What is the scale of this app? A startup app or massive scale?
Interviewer: It should support 50 million daily active users (DAU).
Candidate: For group chat, what is the group member limit?
Interviewer: A minimum of 100 people
Candidate: What features are important for the chat app? Can it support attachment?
Interviewer: 1 on 1 chat, group chat, online indicator. The system should support text messages.
Candidate: Is there a message size limit?
Interviewer: Yes, it should be long enough for less than 100,000 characters long.
Candidate: Is end-to-end encryption required?
Interviewer: Not required for now but we will discuss that if time allows.
Candidate: How long shall we store the chat history?
Interviewer: Forever.

In the interview, focus on designing a chat app that supports the business requirements, with an emphasis on the following features:

- 1 on 1 chat with low latency
- Small group chat (max of 100 people)
- Online presence
- Multiple device support. The same account can be logged in to multiple accounts at the same time.
- Push notifications.

It is also important to agree on the design scale. We will design a system that supports 50 million DAU.

Step 2 - Propose high-level design and get buy-in

To develop a high quality design, we should have a basic knowledge of how client and server communicate. In a chat system, clients can either talk to another client or to a server. For instance, clients in two consumer apps, it asks for the recipient IOT protocol, which is the most common web protocol. In this scenario, the client opens a HTTP connection with the chat service and sends the message, informing the receiver to pick up the message. The receiver client may keep a HTTP connection open to receive a message, but it does not have a persistent connection with the chat service. In other words, the receiver client has to poll HTTP servers. HTTP is fine option on the sender side, and many popular chat applications such as Facebook, IG and Twitter use HTTP mostly to send messages.

The sender client opens a HTTP connection to send a message to the receiver, and keeps the connection open to send messages from the server. Once the receiver is online, the chat service notifies the receiver client through a long polling, and WebSocket. These are important techniques widely used in system design interviews so let us examine each of them.

Polling

Polling is a technique allowing a client ask the frequently ask questions, and the server it does was messages available. Depending on polling frequency, polling could be costly. It could consume precious server resources in a manner that offers no an answer most of the time.

Figure 2

Long polling

Because polling could be inefficient, the next proposeness is long polling (Figure 3).

Figure 3

In long polling, a client holds the connection open until there are actually new messages available or the client timeout has been reached. Once the client receives new messages, it immediately sends another request to the server, restarting the process. Long polling has a few drawbacks:

- Sending messages from other clients.
- Force the light recipients for each message sent/ step the messages to the recipients.
- It is ineffect in a mid-size, mid-frequency chat system.
- Server has to keep connections for every client recently connected to it, which could be inefficient depending on the server implementation.
- Multiple device support. The same account can be logged in to multiple accounts at the same time.
- Push notifications.

It is also inefficient to agree in the design scale. We will design a system that supports 50 million DAU.

Step 2 - Propose high-level design and get buy-in

To develop a high quality design, we should have a basic knowledge of how client and server communicate. In a chat system, clients can either talk to another client or to a server. For instance, clients in two consumer apps, it asks for the recipient IOT protocol, which is the most common web protocol. In this scenario, the client opens a HTTP connection with the chat service and sends the message, informing the receiver to pick up the message. The receiver client may keep a HTTP connection open to receive a message, but it does not have a persistent connection with the chat service. In other words, the receiver client has to poll HTTP servers. HTTP is fine option on the sender side, and many popular chat applications such as Facebook, IG and Twitter use HTTP mostly to send messages.

The sender client opens a HTTP connection to send a message to the receiver, and keeps the connection open to send messages from the server. Once the receiver is online, the chat service notifies the receiver client through a long polling, and WebSocket. These are important techniques widely used in system design interviews so let us examine each of them.

Polling

Polling is a technique allowing a client ask the frequently ask questions, and the server it does was messages available. Depending on polling frequency, polling could be costly. It could consume precious server resources in a manner that offers no an answer most of the time.

Figure 2

Long polling

Because polling could be inefficient, the next proposeness is long polling (Figure 3).

Figure 3

In long polling, a client holds the connection open until there are actually new messages available or the client timeout has been reached. Once the client receives new messages, it immediately sends another request to the server, restarting the process. Long polling has a few drawbacks:

- Sending messages from other clients.
- Force the light recipients for each message sent/ step the messages to the recipients.
- It is ineffect in a mid-size, mid-frequency chat system.
- Server has to keep connections for every client recently connected to it, which could be inefficient depending on the server implementation.
- Multiple device support. The same account can be logged in to multiple accounts at the same time.
- Push notifications.

It is also inefficient to agree in the design scale. We will design a system that supports 50 million DAU.

WebSocket

WebSocket is the most common solution for sending server initiated updates to a client. Figure 4 shows how it works.

Figure 4

WebSocket is a TCP based protocol. It is first established and persistent. It starts its life as a HTTP connection and could be upgraded to WebSocket connection. Messages can be sent from the server or the client. WebSocket is a powerful technique that allows updates to client. WebSocket connections generally work even behind firewalls. It is also easy to set up and it is a TCP based protocol and also used by HTTP/HTTPS connection.

Earlier we said that it is better to use HTTP for the protocol to use, but since WebSocket is bidirectional, there is no strong technical reason not to use it when for anything. Figure 5 shows how WebSocket can be used for both sender and receiver clients.

Figure 5

By using WebSocket for both sending and receiving, it simplifies the design and makes implementation on both client and server more straightforward. Since WebSocket connections are persistent, efficient connection management is critical on the server side.

High-level design

Just now we mentioned that WebSocket was chosen as the main communication protocol between the client and server for its bidirectional communication. It is important to note that everything else can be built on top of WebSocket. In a basic chat features app, we might, over profiles, and of a chat application could use the traditional client server model. Figure 6 shows the high-level design of a chat system.

Figure 6

In Figure 6, the client contains a persistent WebSocket connection to a chat server for real-time messaging.

- Chat service handles message sending/receiving.
- Presence servers manage online/offline status.
- API servers handle everything including user sign-up, change profile, etc.
- Notification services send push notifications.
- Finally, Persist data service layer is used to store chat history. When an offline user comes online, she will see all the previous chat history.

Storage

Data storage for some servers rarely, and keep up running and third-party integrations complete. Deep down the technical stack is the data layer. Both data types are usually stored in the persistent storage. Relational or non-relational DBs are the most ones to decide on the right type of database to cover relational database or NoSQL database? To make an informed decision, we will examine the data types and read/write patterns.

This type of data varies in a typical chat system. The first is general data, such as user profile, setting, user friends etc. These data are relational in nature and stored in relational database. Replication and sharding are common techniques to satisfy availability and scalability requirements.

The second is unique to chat systems. Data of chat history. It is important to understand the read/write pattern.

- The amount of data is enormous for chat systems. It produces 60 billion chat messages per day (assuming 1 billion messages per day, and each message is stored forever).
- Text based data is usually accessed frequently. Users do not usually look up for old chats.
- Although very recent chat history is stored in most cases, client could pull chat history, search random access of data, such as search, view your mentions, jump to specific messages, etc. These cases should be supported by the data access layer.
- The read to write ratio is about 1:1 for 1 on 1 chat apps.

Selecting the correct storage system that supports all of our use cases is crucial. We recommend two value stores for the following reasons:

- The value stores allow horizontal scaling.
- Value stores provide very low latency to access data.
- Relational database do not have nicely long run (2 or 3 year run). When this indexes grow large, random access is expensive.
- Many value stores are adopted by other present reliable chat applications. For example, both Facebook messenger and Discord use key value stores. Facebook messenger uses HBase DB, and Discord uses Cassandra DB.

Data models

Last was, we talked about using key value stores as our storage layer. The most important data is message data. Let us take a close look.

Message table for 1 on 1 chat

Figure 7 shows the message table for 1 on 1 chat. The primary key is message_id which helps to decide message sequence. We cannot rely on created_at to decide the message sequence because two messages can be created at the same time.

message	
message_id	bigint
message_from	bigint
message_to	bigint
content	text
created_at	timestamp

Figure 7

Message table for group chat

Figure 8 shows the message table for group chat. The composite primary key is channel_id, message_id. Channel and group represent the same meaning here, channel_id is the partition key because all queries in a group chat operation is a channel.

group_message	
channel_id	bigint
message_id	bigint
user_id	bigint
content	text
created_at	timestamp

Figure 8

Message ID

Key to particular message id is an interesting topic worth exploring. Message id carries the responsibility of ensuring the order of messages. To ensure/sort the order of messages, message id must satisfy the following two requirements:

- IDs must be unique.
- IDs should be sortable by time, meaning new ones have higher IDs than old ones.

How can we achieve these two requirements? The first one is easy to do by using the "auto_increment" keyword in MySQL. However, this approach does not work for a distributed environment.

The second approach is to use a global 64 sequence number generator like Snowflake [5]. This is discussed in the "Design a unique ID generator in a distributed system" chapter.

The final approach is to use local sequence number generator. Local means IDs are only unique within a group. The reason why local IDs work is that maintaining message sequence within one-on-one channel or a group channel is sufficient. This approach is easier to implement in our business case.

Step 3 - Design deep dive

In a system design interview, usually you are expected to dive deep into some of the components in the high-level design. For the chat system, service discovery, message flows, and presence/offline indicators are deep dive topics.

Service discovery

The primary role of service discovery is to recommend the best server for client based on the criteria like geographical location, server capacity, etc. Apache Zookeeper [7] is a popular open-source solution for service discovery. It registers all the available chat servers and picks the best chat server for a client based on predefined rules.

Figure 9 shows how service discovery (Zookeeper) works.

Figure 9

1. User A tries to log in to the app.
2. The load balancer sends the login request to API servers.
3. After the backend authenticates the user, service discovery finds the best chat server for user A. In this example, server 2 is chosen and the server info is returned back to user A.
4. User A connects to the server 2 through WebSocket.

Message flows

It is interesting to understand the end-to-end flow of a chat system. In this section, we will explore 1 on 1 chat flow, message synchronization across multiple devices and small group chat flows.

1-on-1 chat flow

Figure 10 explains what happens when User A sends a message to User B.

Figure 10

1. User A sends a chat message to Chat server 1.
2. Chat server 1 obtains a message ID from the ID generator.
3. Chat server 1 sends the message to the message sync queue.
4. The message is stored in a key-value store.
5a. If User B is online, the message is forwarded to Chat server 2 where User B is connected.
5b. If User B is offline, a push notification is sent from push notification servers.
6. Chat server 2 forwards the message to User B. There is a persistent WebSocket connection between User B and Chat server 2.

Message synchronization across multiple devices

Many users have multiple devices. We will explain how to sync messages across multiple devices. Figure 11 shows an example of message synchronization.

Figure 11

In Figure 11, user A has two devices, a phone and a laptop. When User A logs in to the chat app with her phone, it establishes a WebSocket connection with Chat server 1. Similarly, there is a connection between the laptop and Chat server 2.

Each device maintains a variable called cur_max_message_id, which keeps track of the latest message ID on the device. Messages that satisfy the following two conditions are considered as new messages:

- The recipient ID is equal to the currently logged in user ID
- Message ID in the key-value store is larger than cur_max_message_id

With distinct cur_max_message_id on each device, message synchronization is easy as each device can get new messages from the KV store.

Small group chat flow

In comparison to the one-on-one chat, the logic of group chat is more complicated. Figures 12-14 and 15-16 explain the flow.

Figure 12

Figure 13 explains what happens when User A sends a message in a group chat. Assume there are 3 members in the group (User A, User B and User C). First, the message from User A is copied to each group member's message sync queue. Just like 1-on-1 chat the second flow (steps 2-6 in Figure 11), four steps of the flow are the same for a recipient. The delivery flow of a message in a small group chat because

- Each group member has a message sync queue to receive messages from other users.
- Since the group member is small, checking inbox to see if there are any new messages is manageable.

WeChat uses a similar approach, and it works a group is 500 members [8]. However, for groups with a lot of users, sharing a message queue for each member is not acceptable.

On the recipient side, a recipient can receive messages from multiple users. Each recipient has an inbox (message sync queue) which contains messages from different sections. Figure 14 illustrates the design.

Figure 15 explains what happens when User A sends a message in a group chat. Assume there are 3 members in the group (User A, User B and User C). First, the message from User A is copied to each group member's message sync queue. Just like 1-on-1 chat the second flow (steps 2-6 in Figure 11), four steps of the flow are the same for a recipient. The delivery flow of a message in a small group chat because

- Each group member has a message sync queue to receive messages from other users.
- Since the group member is small, checking inbox to see if there are any new messages is manageable.

WeChat uses a similar approach, and it works a group is 500 members [8]. However, for groups with a lot of users, sharing a message queue for each member is not acceptable.

On the recipient side, a recipient can receive messages from multiple users. Each recipient has an inbox (message sync queue) which contains messages from different sections. Figure 16 illustrates the design.

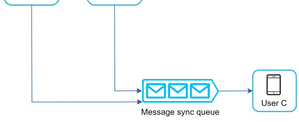

Figure 15

Online presence

An online presence indicator is an essential feature of many chat applications. Usually, you can see a green dot next to a user's profile picture or username. This section explains what happens behind the scenes.

In the high-level design, presence servers are responsible for managing online status and communicating with clients through WebSocket. There are a few flows that will trigger online status change. Let us examine each of them.

User login

The user login flow is explained in the "Service Discovery" section. After a WebSocket connection is built between the client and the real-time service, user A's online status and *last_active_at* timestamp are saved in the KV store. Presence indicator shows the user is online after she logs in.

Figure 16

User logout

When a user logs out, it goes through the user logout flow as shown in Figure 17. The online status is changed to offline in the KV store. The presence indicator shows a user is offline.

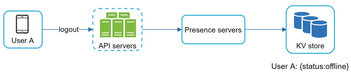

Figure 17

User disconnection

We all wish our internet connection is consistent and reliable. However, that is not always the case; thus, we must address this issue in our design. When a user disconnects from the internet, the persistent connection between the client and server is lost. A naive way to handle user disconnection is to mark the user as offline and change the status to online when the connection re-establishes. However, this approach has a major flaw. It is common for users to disconnect and reconnect to the internet frequently in a short time. For example, network connections can be on and off while a user goes through a tunnel. Updating online status on every disconnect/reconnect would make the presence indicator change too often, resulting in poor user experience.

We introduce a heartbeat mechanism to solve this problem. Periodically, an online client sends a heartbeat event to presence servers. If presence servers receive a heartbeat event within a certain time, say x seconds from the client, a user is considered as online. Otherwise, it is offline.

In Figure 18, the client sends a heartbeat event to the server every 5 seconds. After sending 3 heartbeat events, the client is disconnected and does not reconnect within x = 30 seconds (This number is arbitrarily chosen to demonstrate the logic). The online status is changed to offline.

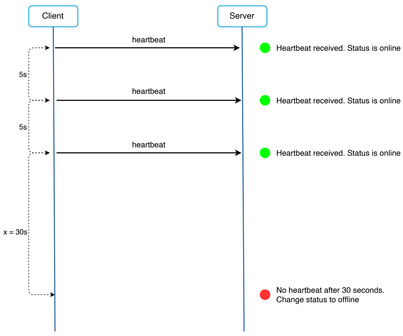

Figure 18

Online status fanout

How do user A's friends know about the status changes? Figure 19 explains how it works. Presence servers use a publish-subscribe model, in which each friend pair maintains a channel. When User A's online status changes, it publishes the event to three channels, channel A-B, A-C, and A-D. Those three channels are subscribed by User B, C, and D, respectively. Thus, it is easy for friends to get online status updates. The communication between clients and servers is through real-time WebSocket.

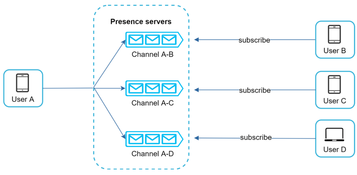

Figure 19

The above design is effective for a small user group. For instance, WeChat uses a similar approach because its user group is capped to 500. For larger groups, informing all members about online status is expensive and time consuming. Assume a group has 100,000 members. Each status change will generate 100,000 events. To solve the performance bottleneck, a possible solution is to fetch online status only when a user enters a group or manually refreshes the friend list.

Step 4 - Wrap up

In this chapter, we presented a chat system architecture that supports both 1-to-1 chat and small group chat. WebSocket is used for real-time communication between the client and server. The chat system contains the following components: chat servers for real-time messaging, presence servers for managing online presence, push notification servers for sending push notifications, key-value stores for chat history persistence and API servers for other functionalities.

If you have extra time at the end of the interview, here are additional talking points:

- Extend the chat app to support media files such as photos and videos. Media files are significantly larger than text in size. Compression, cloud storage, and thumbnails are interesting topics to talk about.
- End-to-end encryption. Whatsapp supports end-to-end encryption for messages. Only the sender and the recipient can read messages. Interested readers should refer to the article in the reference materials [9].
- Caching messages on the client-side is effective to reduce the data transfer between the client and server.
- Improve load time. Slack built a geographically distributed network to cache users' data, channels, etc. for better load time [10].
- Error handling.
- The chat server error. There might be hundreds of thousands, or even more persistent connections to a chat server. If a chat server goes offline, service discovery (Zookeeper) will provide a new chat server for clients to establish new connections with.
- Message resent mechanism. Retry and queueing are common techniques for resending messages.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Reference materials

[1] Erlang at Facebook:
https://www.erlang-factory.com/upload/presentations/31/EugeneLetuchy-ErlangatFacebook.pdf

[2] Messenger and WhatsApp process 60 billion messages a day:
https://www.theverge.com/2016/4/12/11415198/facebook-messenger-whatsapp-number-messages-vs-sms-f8-2016

[3] Long tail: https://en.wikipedia.org/wiki/Long_tail

[4] The Underlying Technology of Messages:
https://www.facebook.com/notes/facebook-engineering/the-underlying-technology-of-messages/454991608919/

[5] How Discord Stores Billions of Messages:
https://discord.com/blog/how-discord-stores-billions-of-messages

[6] Announcing Snowflake: https://blog.twitter.com/engineering/en_us/a/2010/announcing-snowflake.html

[7] Apache ZooKeeper: https://zookeeper.apache.org/

[8] From nothing: the evolution of WeChat background system (Article in Chinese):
https://www.infoq.cn/article/the-road-of-the-growth-weixin-background

[9] End-to-end encryption: https://faq.whatsapp.com/en/android/28030015/

[10] Flannel: An Application-Level Edge Cache to Make Slack Scale:
https://slack.engineering/flannel-an-application-level-edge-cache-to-make-slack-scale-b8a6400e2f6b

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.